

```

65.             # Fire spreads to neighboring trees:
66.             if forest.get((x + ix, y + iy)) == TREE:
67.                 nextForest[(x + ix, y + iy)] = FIRE
68.             # The tree has burned down now, so erase it:
69.             nextForest[(x, y)] = EMPTY
70.         else:
71.             # Just copy the existing object:
72.             nextForest[(x, y)] = forest[(x, y)]
73.     forest = nextForest
74.
75.     time.sleep(PAUSE_LENGTH)
76.
77.
78. def createNewForest():
79.     """Returns a dictionary for a new forest data structure."""
80.     forest = {'width': WIDTH, 'height': HEIGHT}
81.     for x in range(WIDTH):
82.         for y in range(HEIGHT):
83.             if (random.random() * 100) <= INITIAL_TREE_DENSITY:
84.                 forest[(x, y)] = TREE # Start as a tree.
85.             else:
86.                 forest[(x, y)] = EMPTY # Start as an empty space.
87.     return forest
88.
89.
90. def displayForest(forest):
91.     """Display the forest data structure on the screen."""
92.     bext.goto(0, 0)
93.     for y in range(forest['height']):
94.         for x in range(forest['width']):
95.             if forest[(x, y)] == TREE:
96.                 bext.fg('green')
97.                 print(TREE, end='')
98.             elif forest[(x, y)] == FIRE:
99.                 bext.fg('red')
100.                print(FIRE, end='')
101.            elif forest[(x, y)] == EMPTY:
102.                print(EMPTY, end='')
103.        print()
104.        bext.fg('reset') # Use the default font color.
105.        print('Grow chance: {}'.format(GROW_CHANCE * 100), end='')
106.        print('Lightning chance: {}'.format(FIRE_CHANCE * 100), end='')
107.        print('Press Ctrl-C to quit.')
108.
109.
110. # If this program was run (instead of imported), run the game:
111. if __name__ == '__main__':
112.     try:
113.         main()
114.     except KeyboardInterrupt:
115.         sys.exit() # When Ctrl-C is pressed, end the program.

```
